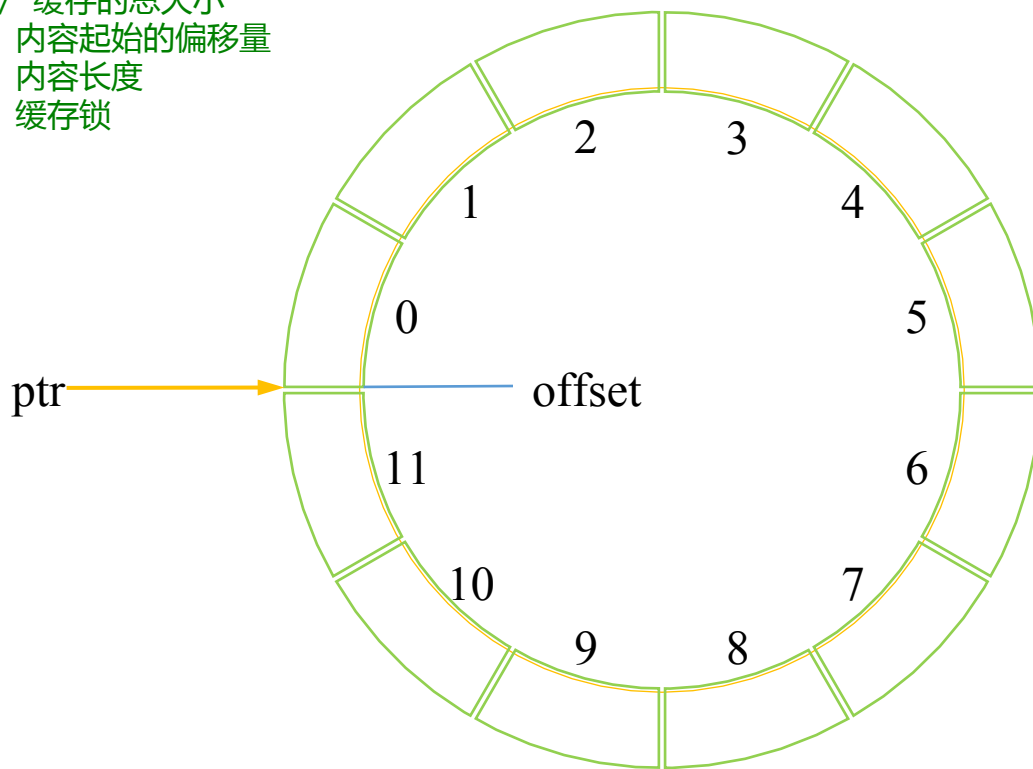


```
typedef struct BufferStruct
{
    unsigned char *ptr;    // 缓存指针
    int total_size;        // 缓存的总大小
    int offset;            // 内容起始的偏移量
    int len;               // 内容长度
    pthread_mutex_t lock;  // 缓存锁
} Buffer;
```

Buffer实例

ptr	字符指针
total_size	12
offset	0
len	0
lock	缓冲锁

环形缓冲区



初始化缓冲区

通过app_buffer_init() 初始化缓冲区，
每个槽可以存放一个char类型数据，

假设结构如左图所示

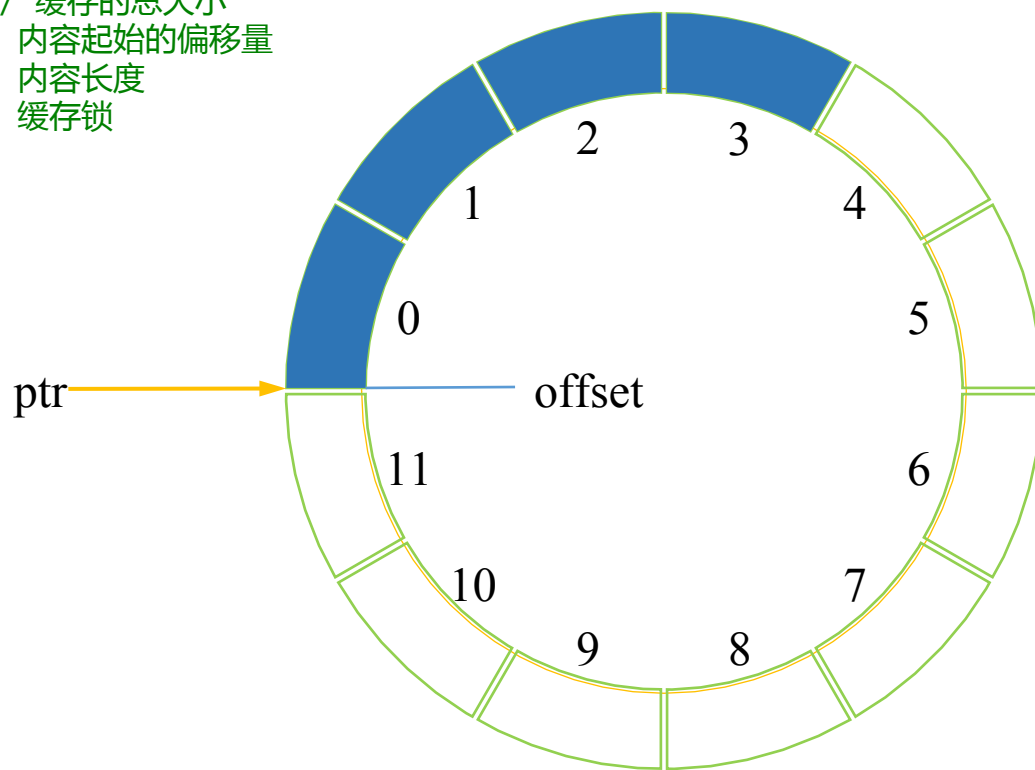
ptr指向偏移量为0的位置，不会变
total_size为12，不会变
offset和len均为0，会变化

```
typedef struct BufferStruct
{
    unsigned char *ptr;    // 缓存指针
    int total_size;        // 缓存的总大小
    int offset;            // 内容起始的偏移量
    int len;               // 内容长度
    pthread_mutex_t lock;  // 缓存锁
} Buffer;
```

Buffer实例

ptr	字符指针
total_size	12
offset	0
len	4
lock	缓冲锁

环形缓冲区



向缓冲区第一次写入数据

通过app_buffer_write()写入长度为4个字节的数据

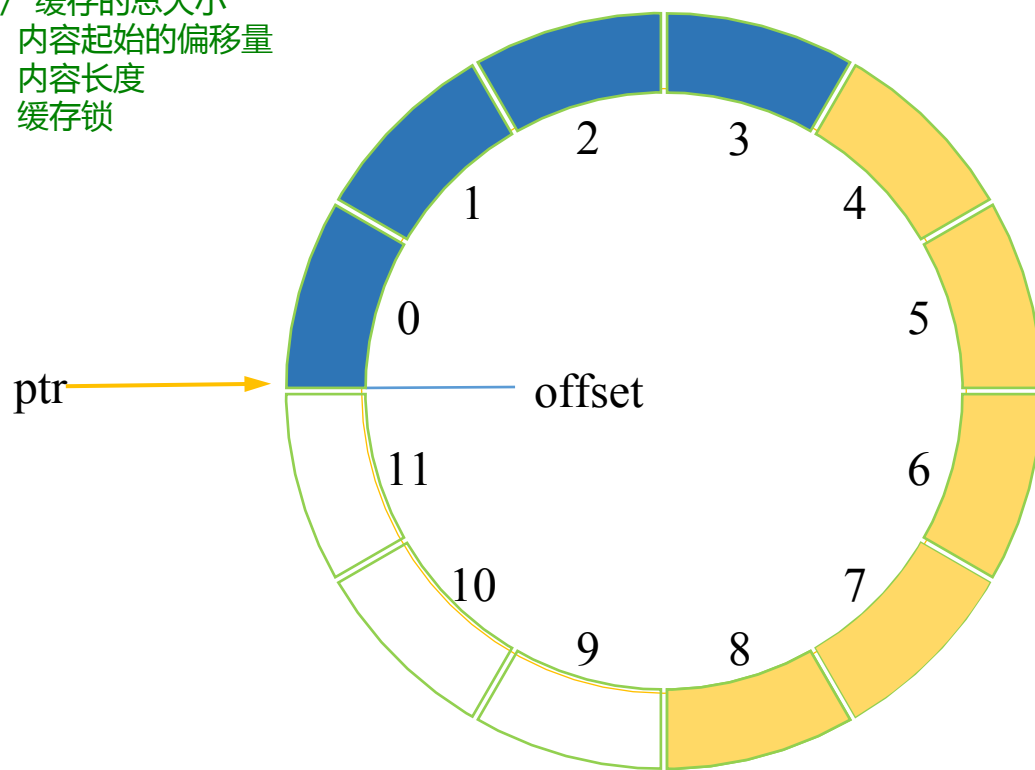
offset依然为0，
len从0变为4

```
typedef struct BufferStruct
{
    unsigned char *ptr;    // 缓存指针
    int total_size;        // 缓存的总大小
    int offset;            // 内容起始的偏移量
    int len;               // 内容长度
    pthread_mutex_t lock;  // 缓存锁
} Buffer;
```

Buffer实例

ptr	字符指针
total_size	12
offset	0
len	9
lock	缓冲锁

环形缓冲区



向缓冲区第二次写入数据

通过app_buffer_write()写入长度为5个字节的数据

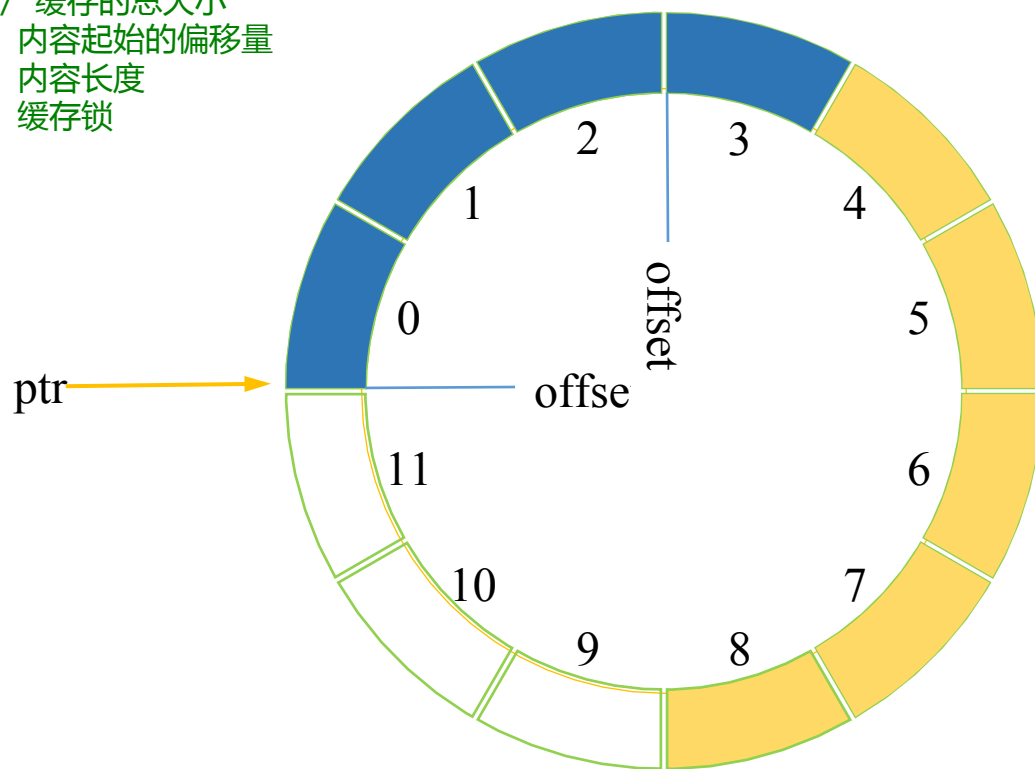
offset依然为0，
len从4变为9

```
typedef struct BufferStruct
{
    unsigned char *ptr;    // 缓存指针
    int total_size;        // 缓存的总大小
    int offset;            // 内容起始的偏移量
    int len;               // 内容长度
    pthread_mutex_t lock;  // 缓存锁
} Buffer;
```

Buffer实例

ptr	字符指针
total_size	12
offset	4
len	5
lock	缓冲锁

环形缓冲区



从缓冲区第一次读出数据

通过app_buffer_read()读出长度为4个字节的数据

offset从0变为4,
len从9变为5

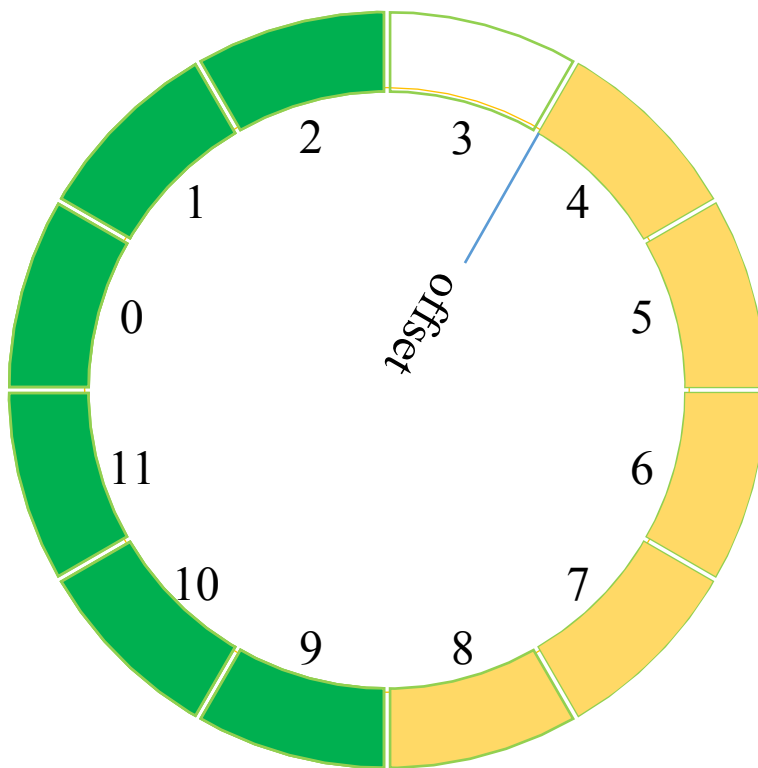
```
typedef struct BufferStruct
{
    unsigned char *ptr;    // 缓存指针
    int total_size;        // 缓存的总大小
    int offset;            // 内容起始的偏移量
    int len;               // 内容长度
    pthread_mutex_t lock;  // 缓存锁
} Buffer;
```

Buffer实例

ptr	字符指针
total_size	12
offset	4
len	11
lock	缓冲锁

ptr →

环形缓冲区



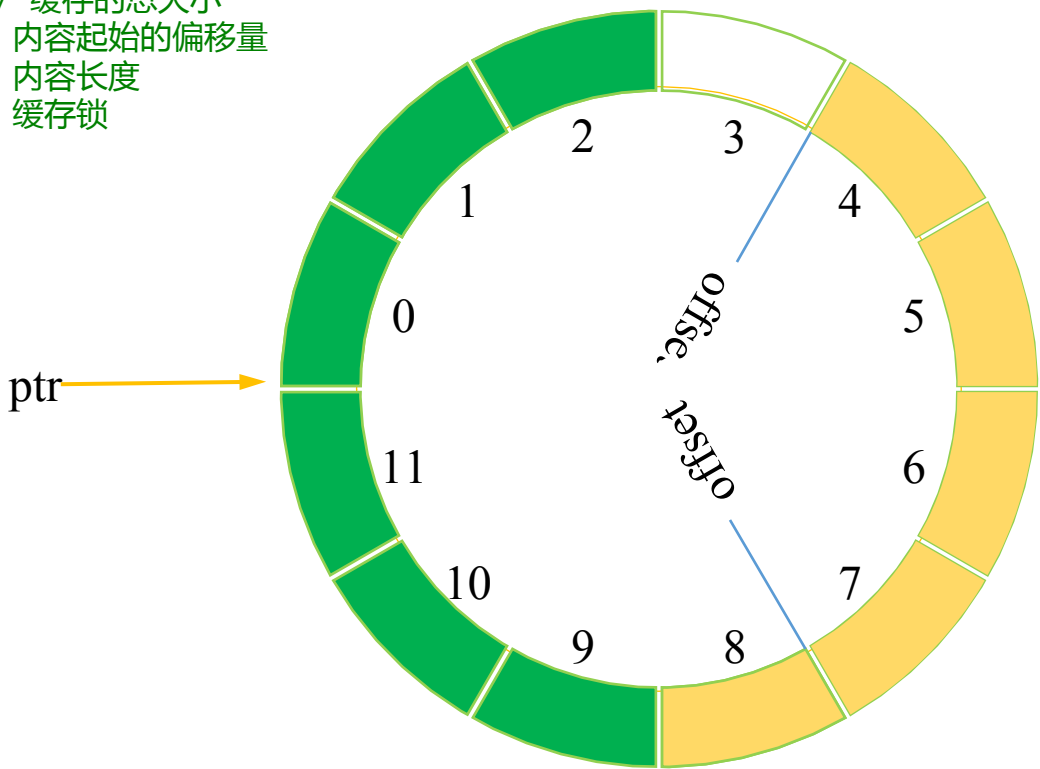
从缓冲区第三次写入数据

通过app_buffer_write()写入长度为6个字节的数据

offset还是4，
len从5变为11

注意：这次实质上需要进入二次连续写入
一次写入为：9, 10, 11
另一次写入为：0, 1, 2

环形缓冲区



从缓冲区第二次读出数据

通过app_buffer_read()读出长度为5个字节的数据

offset从4变为9,
len从11变为6

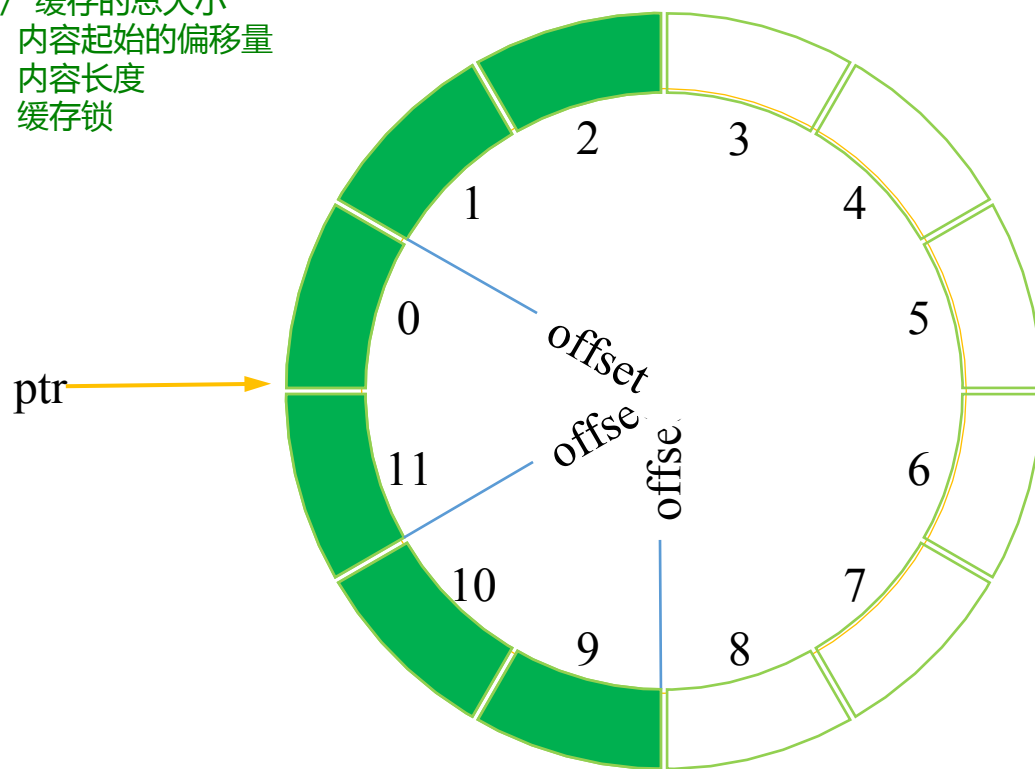
Buffer实例	
ptr	字符指针
total_size	12
offset	9
len	6
lock	缓冲锁

```
typedef struct BufferStruct
{
    unsigned char *ptr;    // 缓存指针
    int total_size;        // 缓存的总大小
    int offset;            // 内容起始的偏移量
    int len;               // 内容长度
    pthread_mutex_t lock;  // 缓存锁
} Buffer;
```

Buffer实例

ptr	字符指针
total_size	12
offset	3
len	0
lock	缓冲锁

环形缓冲区



从缓冲区第三次读出数据

通过app_buffer_read()读出长度为6个字节的数据

offset从9变为3,
len从6变为0

注意：这次实质上需要进入二次连续读出
一次读出为：9, 10, 11
另一次读出为：0, 1, 2